

Relatório Técnico | Sistema de Gerenciamento de Fila Hospitalar

Disciplina: Estruturas de Dados e Algoritmos

Turma: 1CCPR

Grupo:

* Pedro Ricardo de Almeida | 567056

* Agatha Carolina Rodrigues | 568352

* Arthur Berthi | 568017

* João Dyonisio Rocha | 567087

* Nicolas Natal | 568230

1. Escolha das Estruturas de Dados

O sistema combina quatro estruturas, cada uma selecionada pela adequação semântica ao problema que resolve.

Lista Encadeada Simples — primitiva compartilhada

Base de todas as demais estruturas. Cada nó armazena `id`, `nome` e ponteiro `next`.

Por quê: tamanho dinâmico (número de pacientes é imprevisível), inserção/remoção nas extremidades em $O(1)$ e suporte a remoção por ID arbitrário. Arrays exigiriam limite fixo e deslocamento $O(n)$ na remoção do início.

Fila FIFO — atendimento por ordem de chegada

Dois ponteiros `start` (frente) e `end` (último elemento) sobre a lista encadeada.

Por quê: a política FIFO espelha diretamente o critério de justiça hospitalar. O ponteiro `end` reduz `enqueue` de $O(n)$ para **$O(1)$** — crítico em sistemas com alta taxa de chegada. Priorização e cancelamento usam `rm_by_id` + `fix_end` ($O(n)$), aceitos por serem operações raras.

Pilha LIFO — histórico de atendidos (máx. 10)

Ponteiro `top` para o nó mais recente; campo `size` mantido incrementalmente.

Por quê: LIFO exhibe o atendimento mais recente no topo, que é onde o operador espera encontrá-lo. O limite de 10 entradas é uma decisão de domínio — históricos longos têm utilidade decrescente. `push_limited` descarta o registro mais antigo quando cheia.

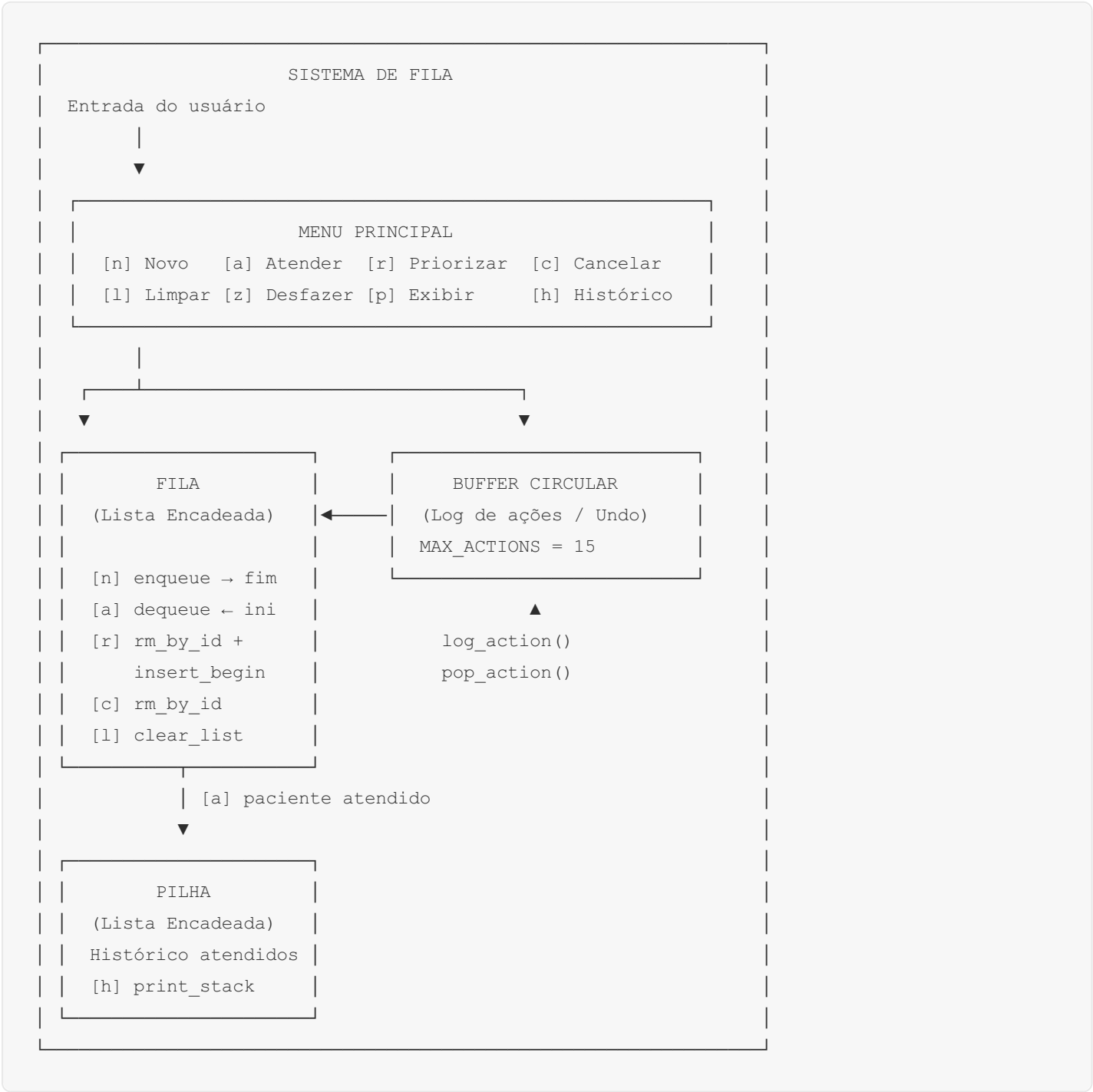
Buffer Circular estático — log de undo (máx. 15 ações)

Array fixo com aritmética modular; sem alocação dinâmica.

Por quê: o limite de 15 ações é fixo e imutável. Arrays superam listas encadeadas nesse cenário: sem overhead de ponteiros por nó, sem fragmentação de heap, operações $O(1)$ garantidas. A pilha encadeada seria

desnecessariamente flexível para um limite estrito e pequeno.

2. Diagrama de Funcionamento



3. Análise de Complexidade

Operação	Complexidade	Observação
insert_begin	O(1)	Atualiza apenas o ponteiro de cabeça
insert_end	O(n)	Percorre até o fim (sem ponteiro de cauda)

Operação	Complexidade	Observação
<code>rm_start</code>	O(1)	Desencadeia o nó de cabeça
<code>rm_end</code>	O(n)	Percorre até o penúltimo nó
<code>rm_by_id</code>	O(n)	Busca linear por identificador
<code>clear_list</code>	O(n)	n remoções do início
<code>enqueue</code>	O(1)	Ponteiro <code>end</code> elimina travessia
<code>dequeue</code>	O(1)	<code>rm_start</code>
<code>is_queue_empty</code>	O(1)	Verifica <code>start == NULL</code>
<code>queue_cancel</code>	O(n)	<code>rm_by_id + fix_end</code>
<code>queue_prioritize</code>	O(n)	<code>rm_by_id + fix_end + insert_begin</code>
<code>push</code>	O(1)	<code>insert_begin</code>
<code>push_limited</code>	O(n) pior caso	<code>rm_end</code> O(n) quando pilha cheia (máx. 10 nós)
<code>pop</code>	O(1)	<code>rm_start</code>
<code>stack_size</code>	O(1)	Campo <code>size</code> mantido incrementalmente
<code>log_action</code>	O(1) — escrita indexada + aritmética modular	—
<code>pop_action</code>	O(1) — leitura indexada + aritmética modular	—
Undo <code>ENQUEUE</code>	O(n)	<code>rm_end + recalcula end</code>
Undo <code>DEQUEUE</code>	O(1)	<code>insert_begin</code>
Undo <code>PRIORITIZE</code>	O(1)	<code>rm_start + enqueue</code>
Undo <code>CANCEL</code>	O(1)	<code>enqueue</code>
Undo <code>CLEAR</code>	—	Não suportado, pois estado anterior descartado

OBS: O consumo de memória é linear no número de pacientes em espera — o mínimo teórico para qualquer sistema que precise armazenar todos eles. As estruturas auxiliares operam com espaço constante.